9:36 10 27.09 KB/s 4G 11%
docs.chaicode.com/di 6
Chai Docs
On this page Git diff

Git diff

The `git diff` is an informative command that shows the differences between two commits. It is used to compare the changes made in one commit with the changes made in another commit. Git consider the changed versions of same file as two different files. Then it gives names to these two files and shows the differences between them.

How to read the diff

- a -> file A and b -> file B
- ---- indicates the file A
- +++ indicates the file B
- @@ indicates the line number

Here the file A and file B are the same file but different versions.

Git will show you the changes made in the file A and file B. It will also show you the line number where the change occurred along with little

Git will show you the changes made in the file A and file B. It will also show you the line number where the change occurred along with little preview of the change.

Comparing Working Directory and Staging Area

```
git diff
```

This command shows the unstaged changes in your working directory compared to the staging area. This command alone will not show you the changes made in the file A and file B, you need to provide options to show the changes.

Comparing Staging Area with Repository

```
git diff --staged
```

This command shows the changes between your

This command shows the changes between your last commit and the staging area (i.e., changes that are staged and ready to be committed).

Comparing between branches

```
git diff <branch-name-one> <branch-name-t
```

This command compares the difference between two branches. For example:

Another way to compare the difference between two branches is to use the following command:

```
git diff branch-name-one..branch-name-two
```

Comparing Specific Commits:

```
git diff <commit-hash-one> <commit-hash-t
```

This command compares the difference between two commits.

Git Stash

Stash is a way to save your changes in a temporary location. It is useful when you want to make changes to a file but don't want to commit them yet. You can then come back to the file later and apply the changes.

> Conflicting changes will not allow you to switch branches without committing the changes. Another alternative is to use the `git stash` command to save your changes in a temporary location.

```
git stash
```

This command saves your changes in a temporary location. It is like a stack of changes that you can access later.

Naming the stash

You can also name the stash by using the following command:

```
git stash save "work in progress on X fea
```

View the stash list

You can view the list of stashes by using the following command:

```
git stash list
```

Apply the stash

You can apply the stash by using the following command:

When working on a Git project, you may need to temporarily save uncommitted changes without committing them. git stash allows you to do this by saving the changes and restoring a clean working directory.Later, when you want those changes back, you can use git stash apply.
By default, it applies the latest stash (stash@{0}).

```
git stash apply
```

Apply the specific stash

You can apply the specific stash by using the following command:

```
git stash apply stash@{0}
```

Here `stash@{0}` is the name of the stash. You can use the `git stash list` command to get the name of the stash.

Applying and dropping the stash

You can apply and drop the stash by using the following command:

The git stash pop command is used to restore the latest stashed changes to your working directory and remove them from the stash list.

```
git stash pop
```

This command applies the stash and drops it from the stash list.

Drop the stash

Drop the stash

You can drop the stash by using the following command:

```
git stash drop
```

Applying stash to a specific branch

You can apply the stash to a specific branch by using the following command:

```
git stash apply stash@{0} <branch-name>
```

Clearing the stash

You can clear the stash by using the following command:

```
```

```
git stash clear
```

git stash clear ;It empties the entire stash stack, permanently removing all saved stash entries.
gti stash drop:It empties the one stash , permanently removing one saved stash entries.

Git Tags

Tags are a way to mark a specific point in your repository. They are useful when you want to remember a specific version of your code or when you want to refer to a specific commit. Tags are like sticky notes that you can attach to your commits.

Creating a tag

You can create a tag using the following command:

```
git tag <tag-name>
```

This command creates a new tag with the specified name. The tag will be attached to the current commit.

This command creates a new tag with the specified name. The tag will be attached to the current commit.

Create an annotated tag

You can create an annotated tag using the following command:

```
git tag -a <tag-name> -m "Release 1.0"
```

This command creates an annotated tag with the specified name and message. The tag will be attached to the current commit.

List all tags

You can list all tags using the following command:

```
git tag
```

This command lists all the tags in your repository.

This command lists all the tags in your repository.

Tagging a specific commit

You can tag a specific commit using the following command:

```
git tag <tag-name> <commit-hash>
```

Push tags to remote repository

You can push tags to a remote repository using the following command:

```
git push origin <tag-name>
```

A remote repository in Git is a repository hosted on a server or cloud platform (such as GitHub, GitLab, or Bitbucket)

Delete a tag

You can delete a tag using the following command:

```
```

Delete a tag

You can delete a tag using the following command:

```
git tag -d <tag-name>
```

Delete tag on remote repository

You can delete a tag on a remote repository using the following command:

```
git push origin :<tag-name>
```

Conclusion

In this section, we have learned about the different commands related to diff, tags and stash in git. These are not main stream commands but they are very useful in certain situations. By the end of this section, you should have a good